

RAG - Cheat Sheet

RAG (Retrieval-Augmented Generation) grounds a language model in your own data. Instead of relying on what the model memorized in training, you retrieve relevant text from a knowledge base at query time and paste it into the prompt. The model answers from that context, not from memory.

Two problems it solves. The model does not know your private or recent data, and it hallucinates when asked about things it half-remembers. RAG fixes both by fetching the actual source text and asking the model to answer from it. No retraining, and you can cite where each answer came from.

RAG vs the alternatives

When you need a model to know something it does not, there are three paths.

Approach	How	Best when
RAG	Retrieve context at query time, inject into the prompt	Knowledge changes often, needs citations, large corpus
Fine-tuning	Train the model's weights on your data	You need a behavior or format change, not fresh facts
Long context	Stuff the whole corpus into one giant prompt	The corpus is small and fits the context window

RAG wins when knowledge is large, changes often, or needs to be attributable. Fine-tuning teaches style and skills, not facts, and goes stale the day your data changes. Long context is simple but pays for every token on every call and degrades as the window fills.

The pipeline

RAG has two phases. Indexing runs offline, ahead of time. Retrieval and generation run per query.

INDEXING (offline)

documents → chunk → embed → store in vector DB

QUERY (online)

question → embed → search → rerank → build prompt → LLM → answer

Get the indexing phase right and the query phase is easy. Most RAG systems that answer badly are retrieving the wrong chunks, not generating badly.

Chunking

Documents are split into chunks before embedding, because you retrieve and inject chunks, not whole files. Chunk size is the single most impactful knob in the pipeline.

Strategy	How it splits
Fixed-size	Every N tokens, with an overlap window
Recursive	On separators (paragraph, then sentence, then word) until under the limit
Structure-aware	On document structure (Markdown headings, code functions)
Semantic	Group adjacent sentences by embedding similarity

Rules of thumb:

- Start around 256 to 512 tokens per chunk. Too large dilutes the match and wastes prompt tokens. Too small loses the surrounding meaning.
- Overlap 10 to 20 percent so a sentence split across a boundary is not orphaned.
- Respect structure. Splitting mid-sentence or mid-function produces chunks that retrieve poorly.
- Keep a bit of metadata on each chunk (source, title, section). You need it for filtering and citations.

Embeddings

An embedding turns text into a vector of floats that captures meaning. Similar text lands close together in the vector space, so search becomes a nearest-neighbor lookup.

- Common models: OpenAI `text-embedding-3-small` (1536 dims) and `-large` (3072), Cohere `embed-v3`, and open models like `bge-large-en`, `e5`, `nomie-embed`.
- Use the **same model** for indexing and for queries. Vectors from two different models are not comparable.
- Normalize vectors and cosine similarity becomes a dot product, which most stores compute faster.
- Larger dimensions capture more nuance but cost more to store and search. Small is usually enough.

```
from openai import OpenAI
client = OpenAI()

def embed(texts: list[str]) -> list[list[float]]:
    resp = client.embeddings.create(
        model="text-embedding-3-small", input=texts)
    return [d.embedding for d in resp.data]
```

Vector stores

A vector store holds the embeddings and answers nearest-neighbor queries fast.

Store	Note
pgvector	A Postgres extension. Reuse the database you already run
Qdrant, Weaviate, Milvus	Purpose-built vector databases, rich filtering
Pinecone	Fully managed, no ops
Elasticsearch, OpenSearch	Vector plus mature keyword search in one engine

Two things to pick:

- **Index type.** HNSW is a graph index, fast with high recall, heavier on memory. IVF clusters vectors, lighter but needs training and tuning. Flat is exact brute force, fine only for small sets.
- **Distance metric.** Cosine similarity is the default for text. Dot product suits normalized vectors. Match whatever your embedding model was trained for.

Retrieval

The base move is a similarity search for the top-k nearest chunks. Three techniques lift quality well past that.

- **Hybrid search.** Combine dense (embedding) search with sparse keyword search (BM25) and fuse the rankings, usually with reciprocal rank fusion. Dense catches meaning, sparse catches exact tokens like error codes, IDs, and names that embeddings blur.
- **Metadata filtering.** Restrict the search by tenant, date, document type, or access level before ranking. This is also where per-user permissions get enforced.
- **MMR (maximal marginal relevance).** Trade a little relevance for diversity so you do not return five near-identical chunks.

Reranking

Retrieval and reranking are a two-stage funnel, and skipping the second stage is the most common quality miss.

- The embedding search (a bi-encoder) is fast but approximate. It scores query and document separately, so it has good recall and mediocre precision.
- A reranker (a cross-encoder) reads the query and each candidate together and scores the pair. It is slower, so you run it only on the retrieved shortlist.

Retrieve a wide net of 20 to 50 candidates, rerank, keep the top 3 to 5 for the prompt. Cohere Rerank and `bge-reranker` are common choices.

Building the prompt

The retrieved chunks go into the prompt with clear instructions.

```
def build_prompt(question: str, chunks: list[str]) -> str:
    context = "\n\n".join(f"[{i+1}] {c}" for i, c in enumerate(chunks))
    return f"""Answer the question using only the context below.
If the answer is not in the context, say you don't know.
Cite the source number in brackets.

Context:
{context}

Question: {question}"""
```

Three instructions earn their place: answer only from context, admit when the answer is absent, and cite the chunk. The “say you don’t know” line is what stops the model from filling gaps with invention.

The full loop

```
def answer(question: str) -> str:
    q_vec = embed([question])[0]
    candidates = store.search(q_vec, top_k=30)           # dense retrieval
    top = rerank(question, candidates)[:5]              # cross-encoder rerank
    prompt = build_prompt(question, [c.text for c in top])

    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0,                                # grounded, not creative
    )
    return resp.choices[0].message.content
```

`temperature=0` keeps generation deterministic and close to the source. RAG wants a faithful reader, not a creative writer.

Evaluation

You cannot improve a RAG system you do not measure, and the two phases fail differently.

Retrieval quality (did the right chunks come back?):

- `recall@k` : fraction of relevant chunks in the top k.
- `MRR` and `nDCG` : reward putting the right chunk near the top.

Generation quality (was the answer good and grounded?):

- **Faithfulness:** every claim in the answer is supported by the retrieved context. This is the RAG-specific one.
- **Answer relevance:** the answer actually addresses the question.
- **Context precision and recall:** how much of the retrieved context was useful, and how much of the needed context was retrieved.

RAGAS packages these, and an LLM-as-judge scores faithfulness at scale. Build a small labeled question set early and run it on every change.

Advanced patterns

Pattern	What it does
Query rewriting	Rewrite a vague or conversational question into a search-friendly one
HyDE	Generate a hypothetical answer, embed that, and search with it
Multi-query	Fan the question into several phrasings, union the results
Parent-document	Embed small chunks for precision, return the larger parent for context
Agentic RAG	Let the model decide when and what to retrieve, in a loop
GraphRAG	Retrieve subgraphs from a knowledge graph, for multi-hop questions

Reach for these only after the basics (good chunks, hybrid search, a reranker, an eval set) are in place. They add cost and moving parts.

Best practices

- **Fix retrieval before touching the prompt.** Most bad answers are bad retrieval. Inspect the chunks that came back.
- **Build an eval set on day one.** A few dozen questions with known answers turns tuning from guessing into measuring.
- **Use hybrid search.** Pure dense retrieval misses exact identifiers. Pure keyword misses meaning.
- **Always rerank.** The cheapest large quality gain in the pipeline.
- **Instruct the model to say “I don’t know”.** A grounded refusal beats a confident hallucination.

- **Cite sources.** Attribution builds trust and makes wrong answers debuggable.
- **Keep chunk metadata.** Source, section, timestamp, and access scope pay off in filtering and citations.
- **Re-index when documents change.** Stale vectors serve stale answers. Track document versions.

Common gotchas

- **Bad chunking poisons everything downstream.** Chunks split mid-thought retrieve poorly no matter how good the model is.
- **Query and document embeddings must share a model.** Mix models and similarity scores are meaningless.
- **Top-k is not free.** More chunks means more tokens, more cost, and more room for the model to lose the point. Rerank down.
- **Embeddings miss exact terms.** Product codes, version numbers, and rare names need keyword search alongside.
- **Lost in the middle.** Models attend less to the center of a long context. Put the strongest chunks first and last.
- **No “I don’t know” path.** Without the instruction, the model answers even when nothing relevant was retrieved.
- **Skipping evaluation.** Without metrics, every change is a vibe. You will tune in circles.
- **Ignoring access control.** Retrieval must filter by who is asking, or RAG becomes a data-leak channel across tenants and users.
- **Re-embedding cost at scale.** Changing the embedding model means re-embedding the whole corpus. Pick deliberately.
- **Context window is a budget, not a target.** Filling it dilutes the signal and slows the call. Retrieve less, better.

Quick reference

Concept	Key fact
RAG	Retrieve context at query time, generate a grounded answer
Indexing	Offline: chunk, embed, store
Query	Online: embed, search, rerank, prompt, generate
Chunk size	Start 256 to 512 tokens, 10 to 20 percent overlap
Embedding	Text to vector, same model for docs and queries
Vector store	Nearest-neighbor search, HNSW or IVF index
Distance	Cosine for text, dot for normalized vectors
Hybrid search	Dense plus BM25, fused with RRF
Reranking	Cross-encoder over the shortlist, keep top 3 to 5
Prompt rule	Answer from context only, cite, admit ignorance
Temperature	0 for grounded generation
Faithfulness	Answer is supported by retrieved context
RAGAS	Faithfulness, relevance, context precision and recall
HyDE	Search with a hypothetical answer, not the raw query
Access control	Filter retrieval by who is asking
Common failure	Bad retrieval, not bad generation